СУЧАСНІ ТЕХНОЛОГІЇ ПАРАЛЕЛЬНОГО ПРОГРАМУВАННЯ

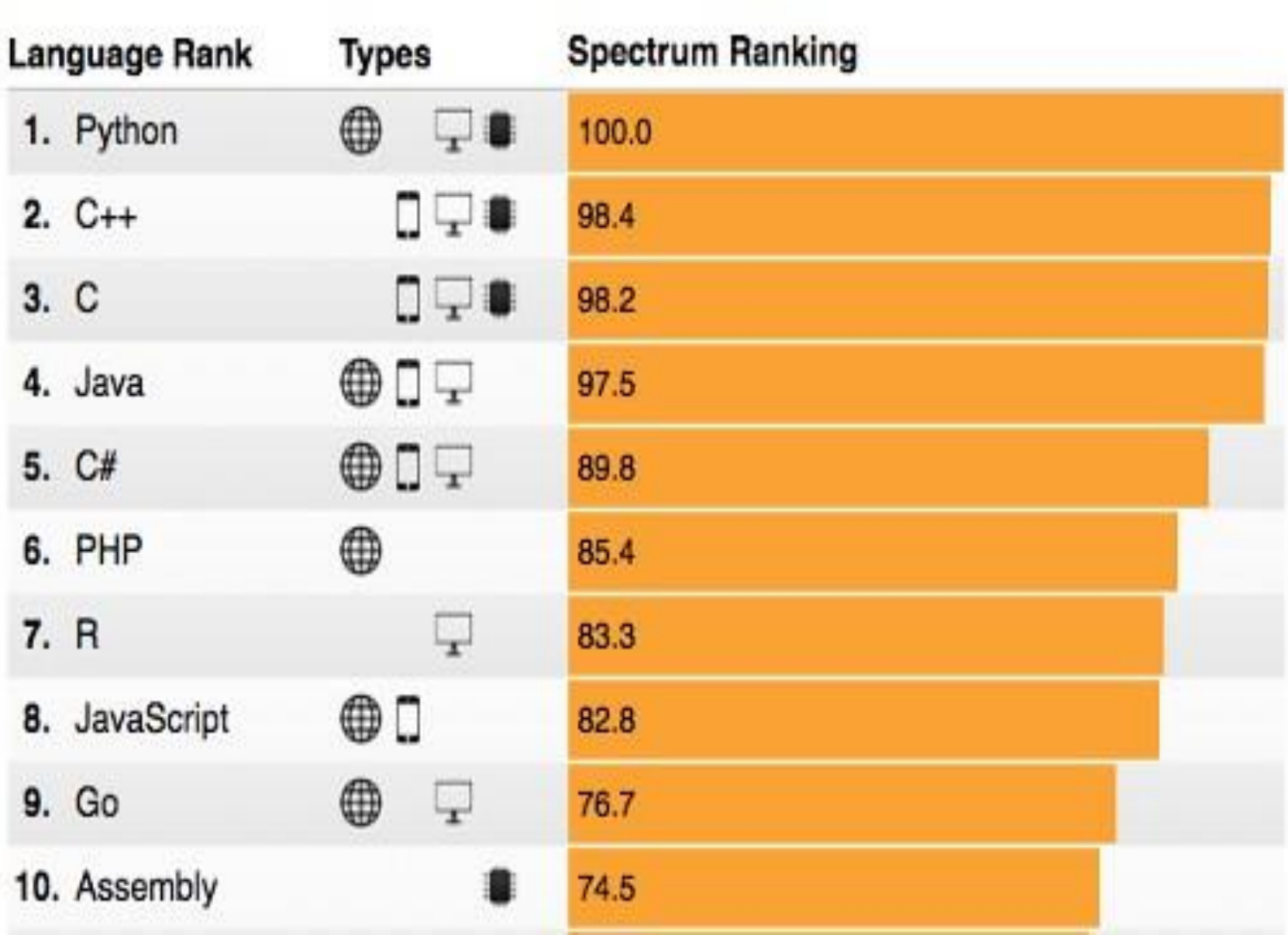
Два шляхи переносимості паралельного ПЗ

- Засоби паралельного програмування застосовуються для різних архітектур мультипроцесорних систем та різних процесорних платформ з використанням різних мов програмування.
- За такої великої розмаїтості з метою уніфікації та переносимості паралельного програмного забезпечення йдуть двома основними шляхами:
  - 1) *розширення існуючих мов* програмування та створення нових компіляторів для них (наприклад від C до C++):
  - 2) *створення бібліотек функцій* та програмних оболонок, що дозволяють, не змінюючи компілятора з базової мови програмування, забезпечувати стандартний інтерфейс програмування.

Переваги обох підходів

- Обидва підходи *не виключають, а доповнюють один одного*, проте другий підхід має ту перевагу, що зберігає можливість використання накопиченого програмного забезпечення.
- Класичним прикладом першого з них є *поява мови програмування Java*, що грунтується на сучасних концепціях об’єктної орієнтованості, мультиплатформної реалізації і підтримки паралелізму.
- Прикладами другого підходу є *сучасні інтерфейси (середовища) паралельного програмування*, що є надбудовами над стандартними компіляторами.
- Вони являють собою *бібліотеки функцій*, за допомогою яких паралельні алгоритми можуть програмуватись *єдиним чином* на багатьох мультипроцесорних системах.

Популярність сучасних мов програмування - 2018



PVM (Parallel Virtual Machine) (1)

- *PVM* була історично першою розробка (початку 90-х рр., США), головною метою було забезпечення можливості паралельного програмування на різнорідних (гетерогенних) мережах комп′ютерів, з′єднаних протоколом TCP/IP.
- PVM являє собою пакет програм, написаних мовою програмування C, що дозволяє розглядати гетерогенну мережу Unix-машин, як *єдиний паралельний комп′ютер* з розподіленою пам′яттю (віртуальну машину).
- До складу віртуальної машини можуть входити як послідовні, так і паралельні комп′ютери.
- Основною одиницею роботи в PVM є задача (task), що як правило, відповідає поняттю процесу у UNIX. Задачі можуть обмінюватись повідомленнями і утворювати таким чином застосунок (application) — що може бути написане мовою C або FORTRAN77.

PVM (Parallel Virtual Machine) (2)

- PVM складається з двох частин: *клієнтської* та серверної.
- Серверна частина – це множина *програм-демонів*, розподілених по комп′ютерам віртуальної машини. Клієнтська частина утворена *бібліотечними функціями PVM*, що є в розпорядженні користувача для підтримки обміну повідомленнями та координації її задач.
- На одному і тому ж обладнанні мережі може існувати *декілька віртуальних машин*, які одночасно можуть виконувати декілька застосувань.
- Для виконання застосунку користувач спочатку породжує демон на своєму комп′ютері, і застосунок може стартувати з будь-якого комп′ютера віртуальної машини.
- В разі виходу з ладу одного з комп′ютерів PVM автоматично реєструє цю подію і виводить цей комп′ютер зі складу віртуальної машини.

PVM (Parallel Virtual Machine) (3)

- Модель взаємодії задач у PVM *не накладає обмежень* ні на кількість, ні на обсяг повідомлень, якими обмінюються задачі. Крім парних взаємодій можливі також *групові операції*: "група – одному" (multicast), "один – групі" (broadcast).
- Модель обміну повідомленнями PVM передбачає також *збереження порядку* проходження повідомлень.
- Отже, особливостями PVM є :
  - підтримка гетерогенних мереж ;
  - динамічне керування ресурсами віртуальної машини.
- Ці особливості є *перевагами PVM*, проте вони спричиняють і *головний недолік PVM* – порівняно невисоку ефективність паралельних обчислень через великі накладні витрати на підтримку своїх переваг.

MPI (Message Passing Interface) (1)

- Так само, як і в PVM, в MPI реалізується підхід розширення відомих мов *за рахунок бібліотеки функцій*, що забезпечують засоби взаємодії паралельних процесів.
- Проте метою MPI є забезпечення високої продуктивності та переносимості застосувань на різних мультипроцесорних платформах.
- На теперішній час MPI є *машинно-незалежним стандартом* для паралельного програмування, що включає понад 100 бібліотек функцій.
- На відміну PVM, яку можна розглядати як *розподілену операційну систему* для гетерогенних мереж, MPI є тільки інтерфейс, розрахований на *однорідні* мультипроцесорні системи.

MPI (Message Passing Interface) (1)

- MPI-програма складається з процесів з унікальними номерами 0..N-1, не має спільної пам′яті і розрахована на MIMD архітектуру паралельної системи.
- У більшості реалізацій MPI кількість процесів є величиною сталою, що визначається при ініціалізації програми. Стандартними мовами програмування для MPI є C та FORTRAN.
- У наступному прикладі використовується декілька головних функцій MPI:
  - include <mpi.h>
  - main (argc, argv){int myrank;
  - MPI_init (&argc, &argv) : /*ініціалізація MPI*/
  - MPI_comm_rank (
  - MPI_comm_world, /*стандартний комунікатор*/
  - myrank); /Номер процесу від 0 до N-1*/
  - if (myrank = = 0) {
  - master () ; } /*керуюча програма*/
  - else {slave ()} /*підлегла програма*/
  - MPI_finalize () ;}

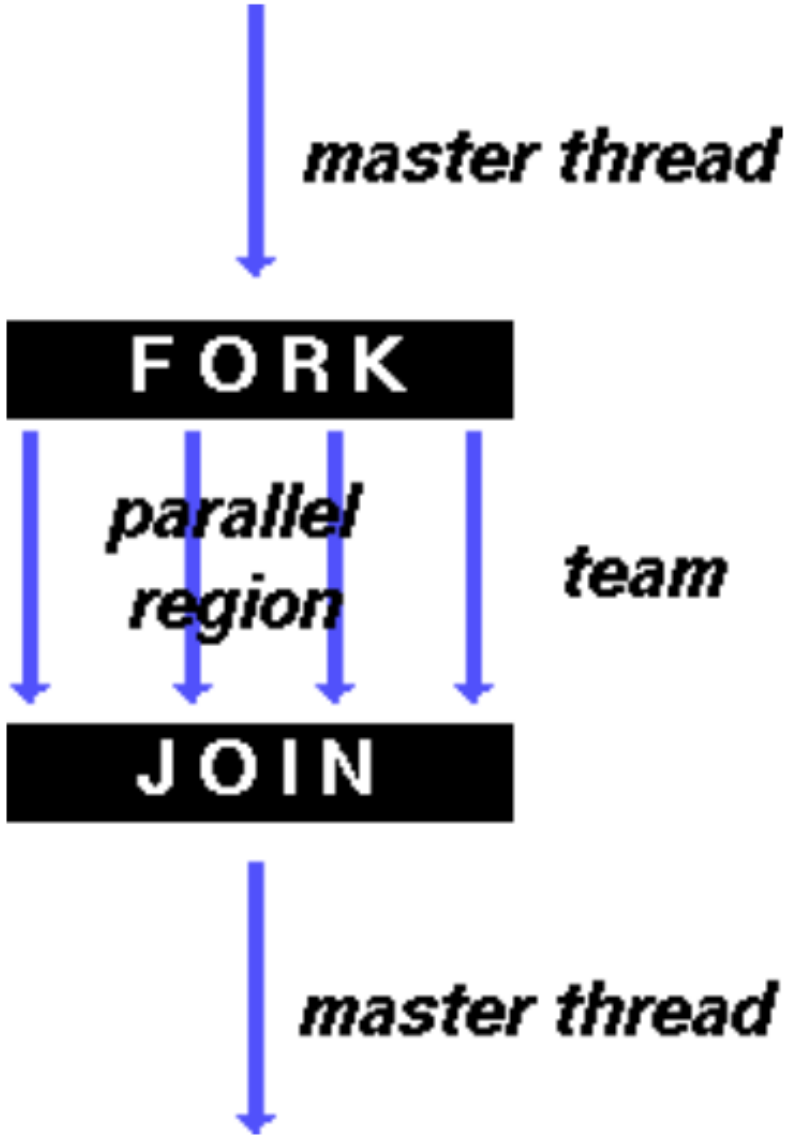
MPI (Message Passing Interface) (2)

- Крім використаних у прикладі 4-х функцій до шістки найуживаніших належать також функції передачі MPI_send і прийому MPI_recv повідомлення.
- Важливим поняттям MPI є поняття комунікатора — тобто середовища, в якому група процесорів може здійснювати обмін повідомленнями.
- Стандартним комунікатором є MPI_comm_world, який визначає універсальне середовище для обмінів — тобто групу процесорів, пронумерованих послідовно від 0 до N-1.
- Так само, як і PVM, MPI має декілька режимів передачі/прийому повідомлень.

OpenMP (Open Multi-Processing)

- Головною метою появи OpenMP як стандарту для різних архітектур і платформ зі *спільною пам'яттю* було:
  - Дати простий, але обмежений набір директив для паралелізації програм
  - Забезпечувати сумісність і можливість нарощуваної (інкрементальної) паралелізації програм.
  - Дати можливість як для дрібнозернистого, так і для крупнозернистого розпаралелювання.
  - Забезпечити підтримку мов програмування Fortran (версії 77, 90 і 95), C, і C ++.

Модель виконання OpenMP



Про дані у розпаралеленій програмі

- У паралельних програмах змінні діляться *на два класи*:
- змінні "Private" видимі *тільки для одного потоку*. Це означає, що:
  - Зміни в значенні змінної локальні і не видні іншим потокам
  - Наприклад, локальна змінна в функції, яка виконується паралельно.
- змінні "Shared" *видима всім потокам*. Це означає, що:
  - Зміни в значенні цієї змінної *видні всім потокам*
  - Ці змінні представляють глобальні дані

Приклад – розпаралелювання циклу (для мови C)

```
#pragma omp parallel shared(a,b)   мітка
```

• {	
• #pragma omp for private(i)	мітка
• for(i=0; i<10000; i++)	
• a[i] = a[i] + b[i];	
• }	прихований бар'єр

Директиви

- Директиви OpenMP з точки зору Фортрана є *коментарями* і починаються з комбінації символів "! $ OMP« (для Фортрана).
- Директиви можна розділити на 3 категорії:
  - визначення паралельної секції,
  - розподіл роботи,
  - синхронізація.
- Кожна директива може мати кілька додаткових *атрибутів - клауз*.
- Окремо визначаються клаузи для призначення класів змінних, які можуть бути атрибутами різних директив.

Простий приклад – обчислення числа π

```
program compute_pi
        parameter (n = 1000)
        integer i
        double precision w,x,sum,pi,f,a
        f(a) = 4.d0/(1.d0+a*a)
        w = 1.0d0/n sum = 0.0d0;
!$OMP PARALLEL DO PRIVATE(x), SHARED(w)
!$OMP& REDUCTION(+:sum)
        do i=1,n
                x = w*(i-0.5d0)
                sum = sum + f(x)
        enddo
        pi = w*sum
        print *,'pi = ',pi
        stop
        end
```